Travessia em Grafos

Estruturas de Dados



UnB | CIC

Busca em Largura

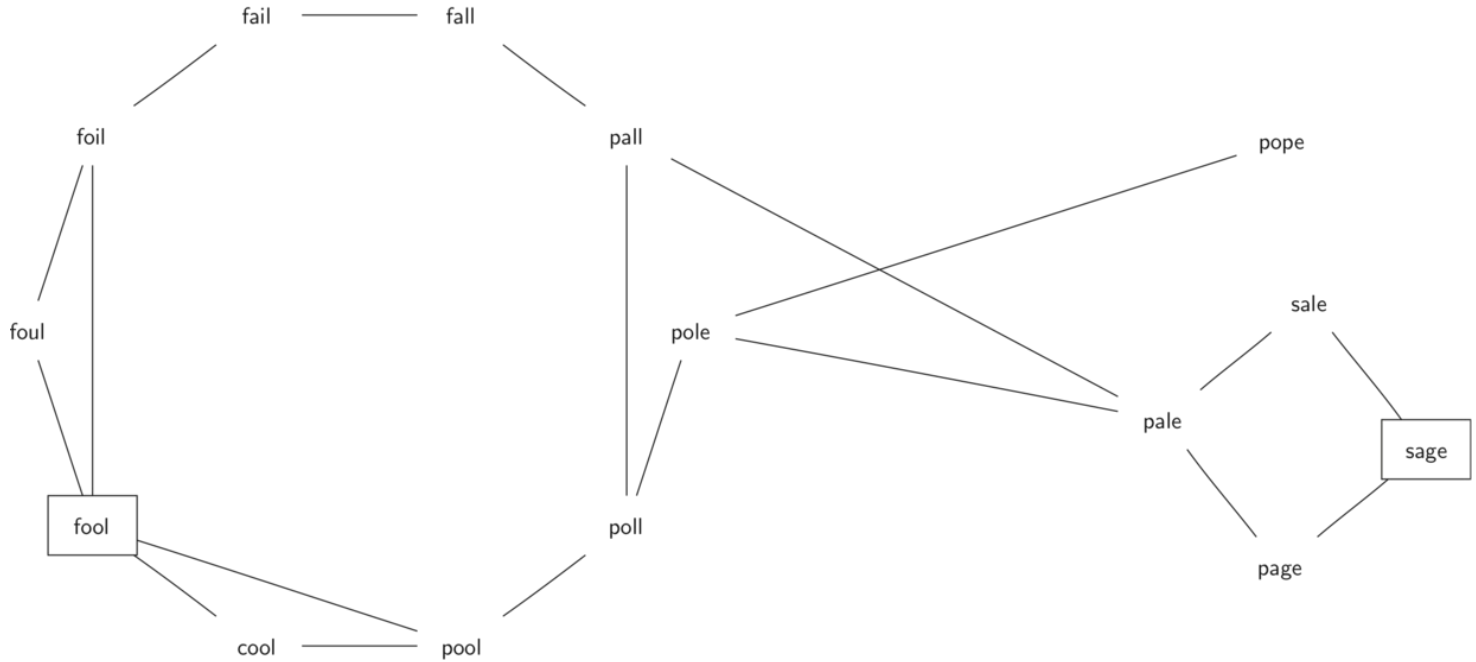
Dado um grafo **G** e um vértice inicial **s**

- Explora arestas no grafo para encontrar todos os vértices em **G** para os quais existe um caminho a partir de **s.**
- Encontra todos os vértices que estão a uma distância **k** de **s** antes de encontrar quaisquer vértices que estão a uma distância **k+1**.

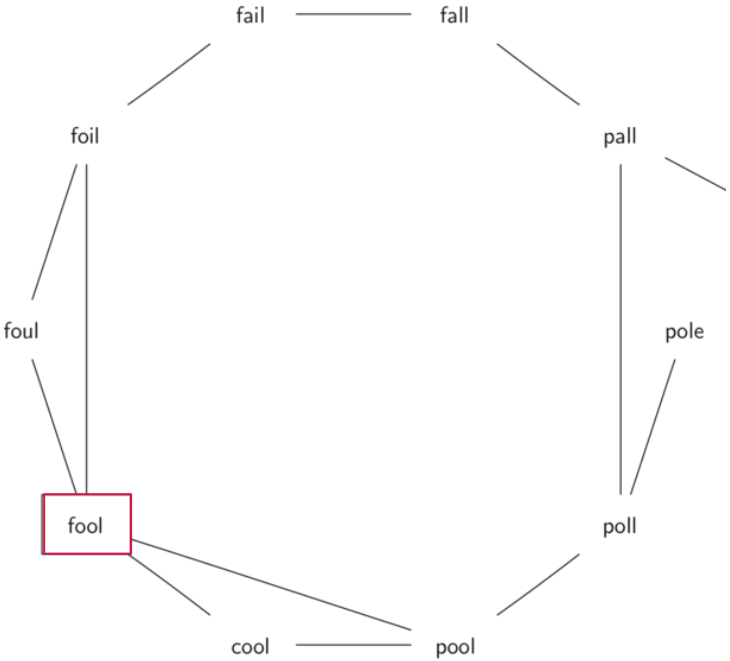
Busca em Largura

- Utiliza a representação de árvore para visualizar os nós do grafo.
- A busca em largura visita os nós filhos antes de visitar os nós netos.
- Os nós são coloridos em branco, cinza ou preto.
  - Um vértice branco é um vértice a ser descoberto.
  - Um vértice cinza é um vértice descoberto.
  - Um vértice preto é um vértice completamente explorado (incluindo seus filhos).

Busca em Largura

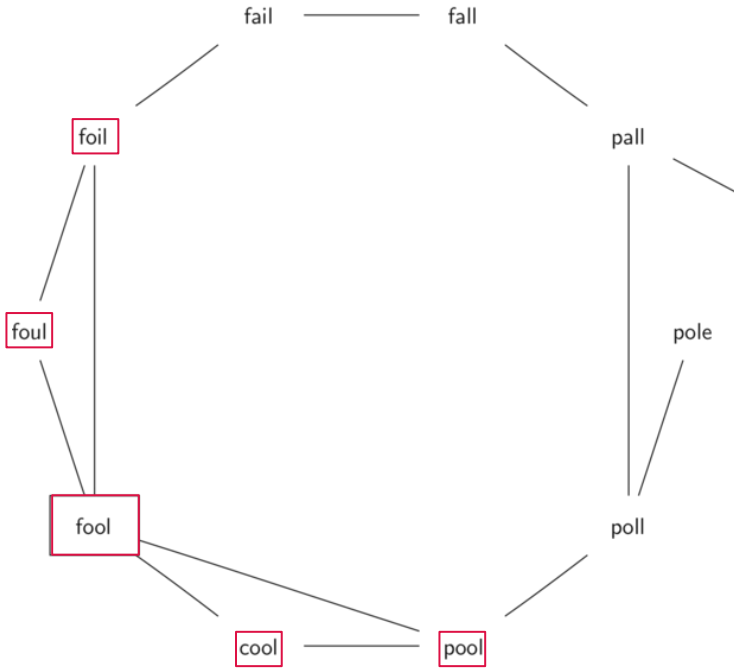


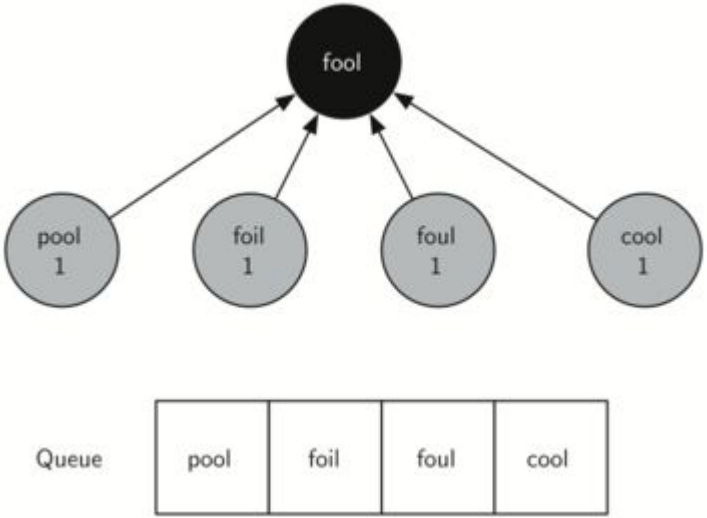
Busca em Largura



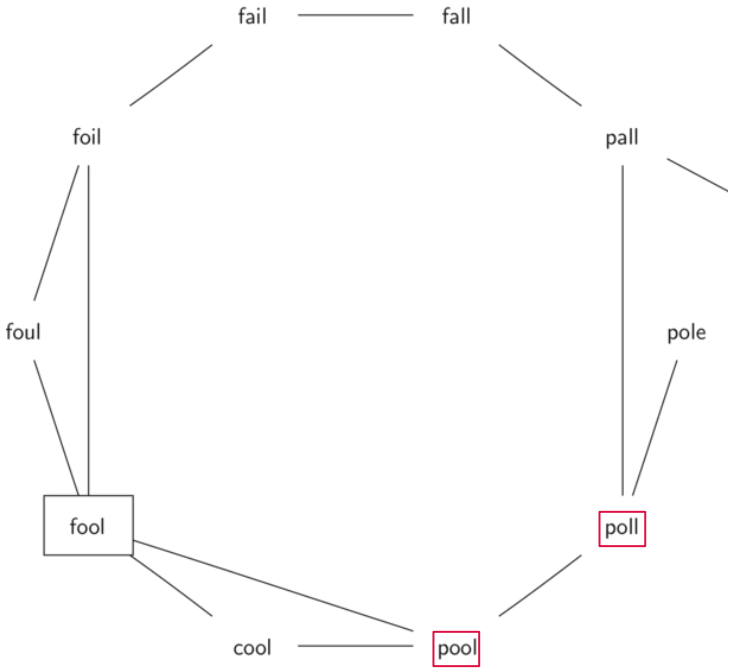
fool

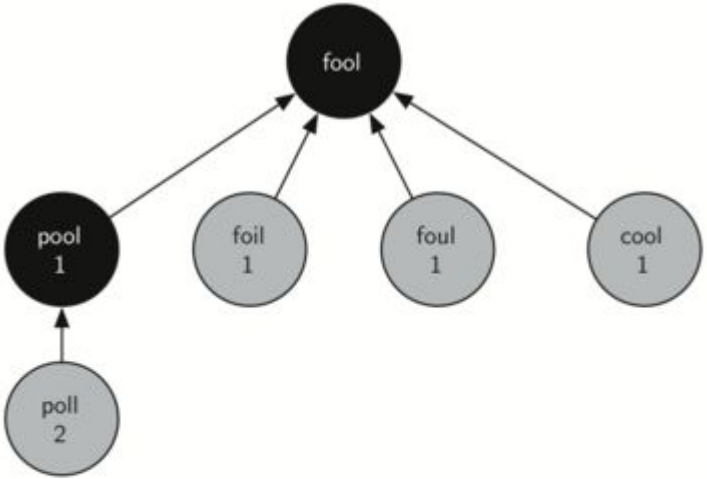
Busca em Largura





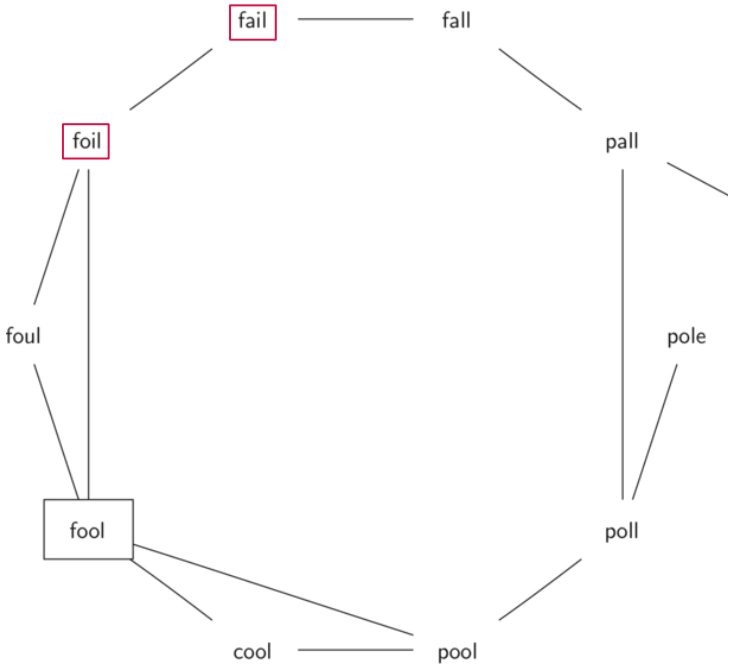
Busca em Largura

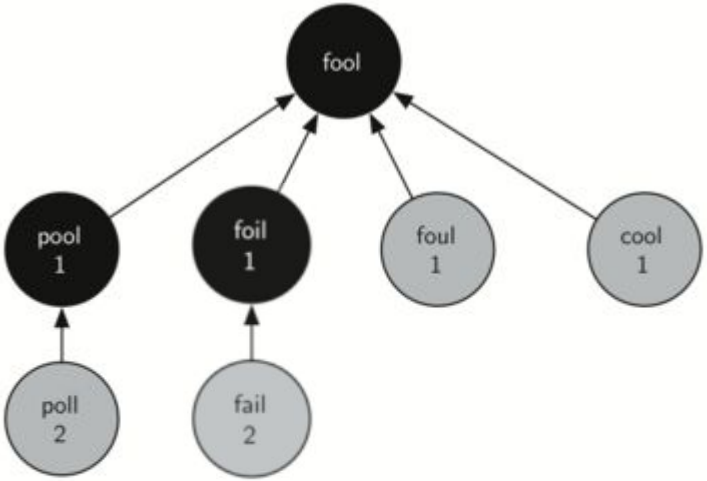






Busca em Largura

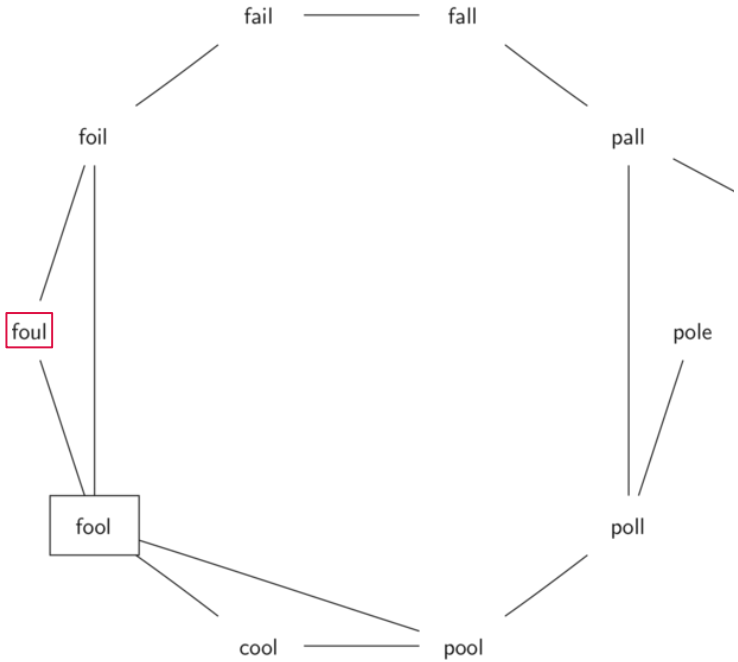


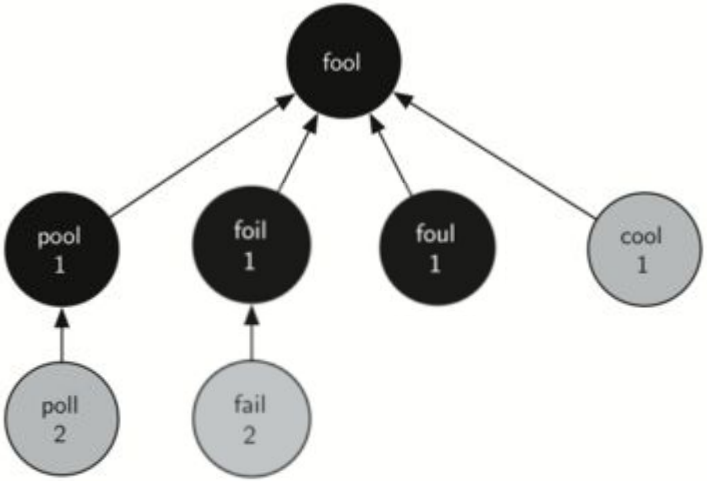


Queue

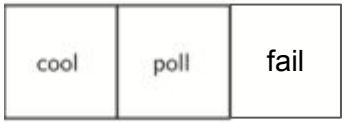


Busca em Largura

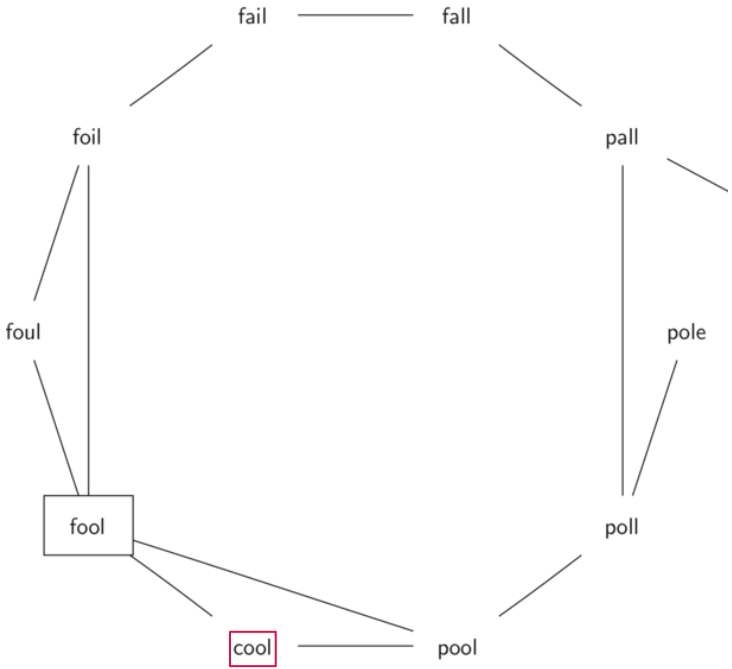


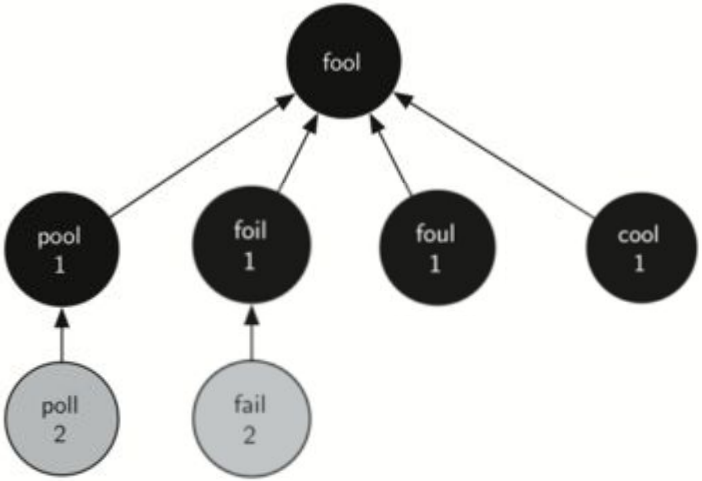


Queue



Busca em Largura

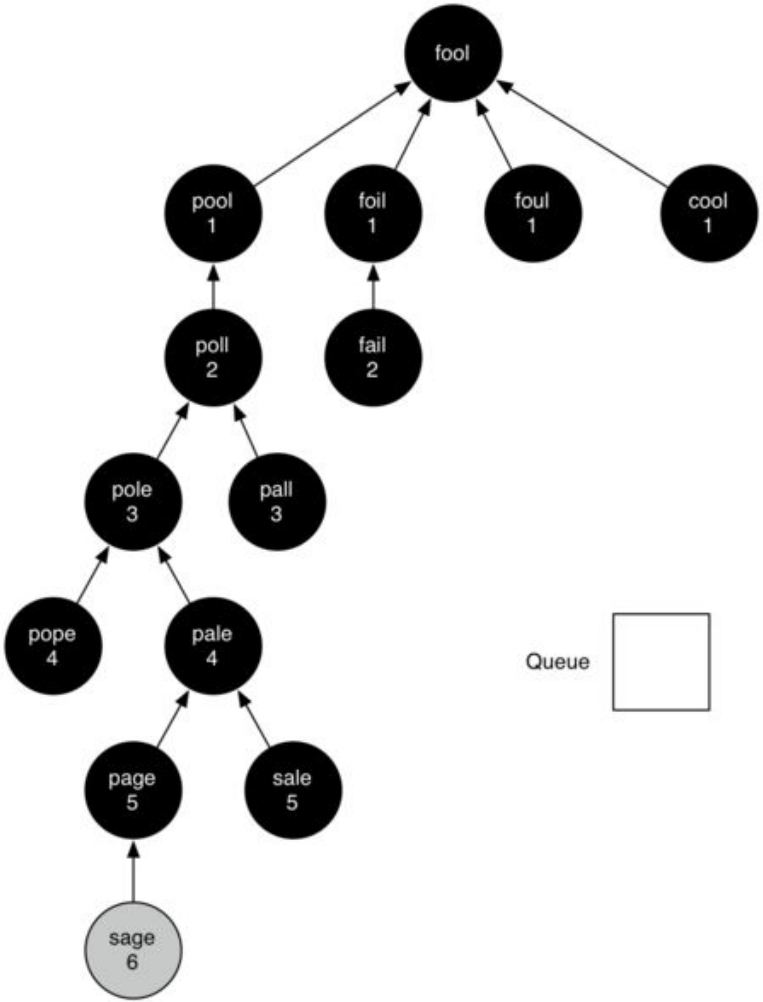




Queue



Busca em Largura



Busca em Largura

```
from pythonds.graphs import Graph, Vertex
from pythonds.basic import Queue

def bfs(start):
    start.setDistance(0)
    start.setPred(None)
    vertQueue = Queue()
    vertQueue.enqueue(start)
    while vertQueue.size() > 0:
        currentVert = vertQueue.dequeue()
        for nbr in currentVert.getConnections():
            if nbr.getColor() == 'white':
                nbr.setColor('gray')
                nbr.setDistance(currentVert.getDistance() + 1)
                nbr.setPred(currentVert)
                vertQueue.enqueue(nbr)
        currentVert.setColor('black')
```

Busca em Largura

```
from pythonds.graphs import Graph, Vertex
from pythonds.basic import Queue

def bfs(start):
    start.setDistance(0)
    start.setPred(None)
    vertQueue = Queue()
    vertQueue.enqueue(start)
    while vertQueue.size() > 0:
        currentVert = vertQueue.dequeue()
        for nbr in currentVert.getConnections():
            if nbr.getColor() == 'white':
                nbr.setColor('gray')
                nbr.setDistance(currentVert.getDistance() + 1)
                nbr.setPred(currentVert)
                vertQueue.enqueue(nbr)
        currentVert.setColor('black')
```

$O(V)$

$O(E)$

Busca em Largura

```
from pythonds.graphs import Graph, Vertex
from pythonds.basic import Queue

def bfs(start):
    start.setDistance(0)
    start.setPred(None)
    vertQueue = Queue()
    vertQueue.enqueue(start)
    while vertQueue.size() > 0:
        currentVert = vertQueue.dequeue()
        for nbr in currentVert.getConnections():
            if nbr.getColor() == 'white':
                nbr.setColor('gray')
                nbr.setDistance(currentVert.getDistance() + 1)
                nbr.setPred(currentVert)
                vertQueue.enqueue(nbr)
        currentVert.setColor('black')
```

$O(V)$

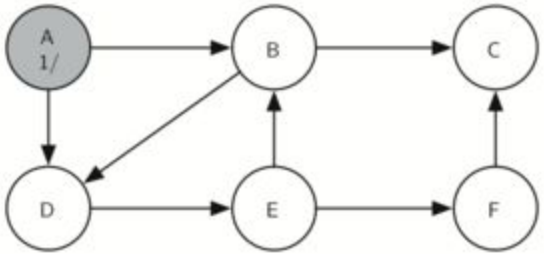
$O(E)$

$O(V+E)$

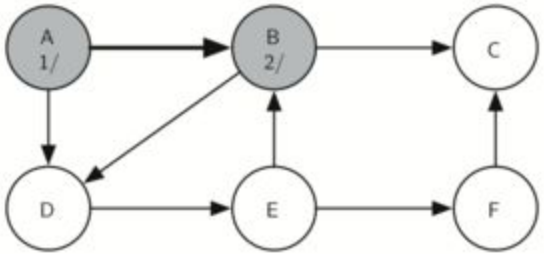
Busca em Profundidade

- Utiliza o conceito de antecessor.
- Os nós são coloridos em branco, cinza ou preto.
- Faz uso de variáveis de tempo de descoberta e tempo de término.
  - O tempo de descoberta é o número de passos antes de um vértice ser encontrado pela primeira vez.
  - O tempo de término é o número de passos antes que um vértice seja colorido de preto.

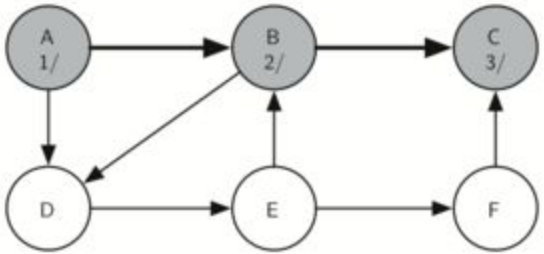
Busca em Profundidade



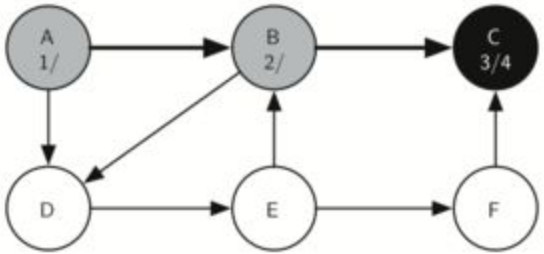
Busca em Profundidade



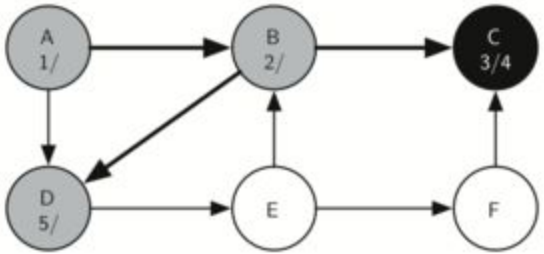
Busca em Profundidade



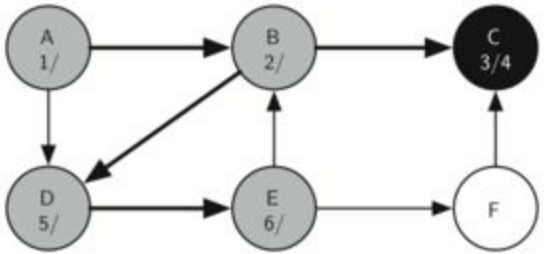
Busca em Profundidade



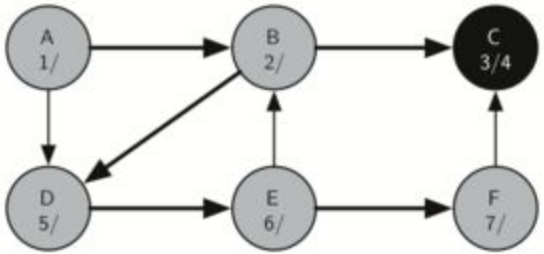
Busca em Profundidade



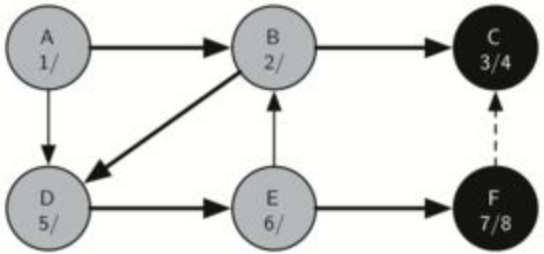
Busca em Profundidade



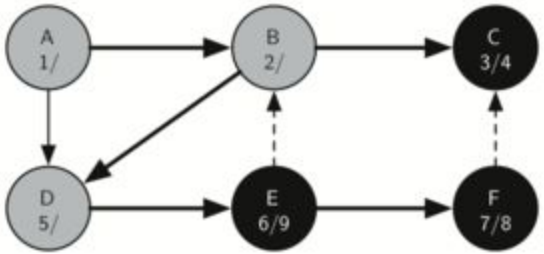
Busca em Profundidade



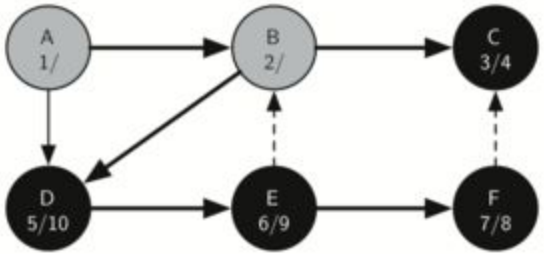
Busca em Profundidade



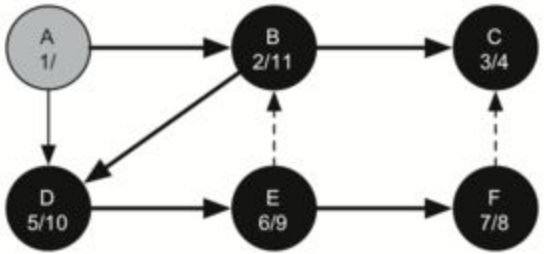
Busca em Profundidade



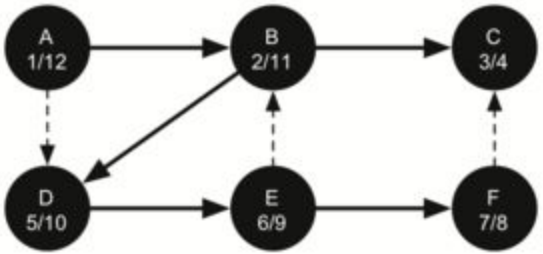
Busca em Profundidade



Busca em Profundidade



Busca em Profundidade



Busca em Profundidade

```
class DFSGraph(Graph):

    def __init__(self):
        super().__init__()
        self.time = 0
```

Busca em Profundidade

```
def dfsvisit(self, startVertex):
    startVertex.setColor('gray')
    self.time += 1
    startVertex.setDiscovery(self.time)
    for nextVertex in startVertex.getConnections():
        if nextVertex.getColor() == 'white':
            nextVertex.setPred(startVertex)
            self.dfsvisit(nextVertex)
    startVertex.setColor('black')
    self.time += 1
    startVertex.setFinish(self.time)
```

Busca em Profundidade

```
def dfsvisit(self, startVertex):
    startVertex.setColor('gray')
    self.time += 1
    startVertex.setDiscovery(self.time)
    for nextVertex in startVertex.getConnections():
        if nextVertex.getColor() == 'white':
            nextVertex.setPred(startVertex)
            self.dfsvisit(nextVertex)
    startVertex.setColor('black')
    self.time += 1
    startVertex.setFinish(self.time)
```

$O(E)$

$O(V)$

Busca em Profundidade

```
def dfsvisit(self, startVertex):
    startVertex.setColor('gray')
    self.time += 1
    startVertex.setDiscovery(self.time)
    for nextVertex in startVertex.getConnections():
        if nextVertex.getColor() == 'white':
            nextVertex.setPred(startVertex)
            self.dfsvisit(nextVertex)
    startVertex.setColor('black')
    self.time += 1
    startVertex.setFinish(self.time)
```

$O(E)$

$O(V)$

$O(V+E)$

Resumo

- Podemos atravessar um grafo usando busca em largura ou profundidade.
- A busca em largura mantém:
  - O estado do nó (conhecido, visitado ou consumido)
- A busca em profundidade mantém:
  - O estado do nó (conhecido, visitado ou consumido)
  - Tempo (descoberta e término)
  - Antecessor (nó)